



IBM QRadar SIEM Connector — Credential Provisioning Guide

Version 0.1.0, 2026-07-16

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	2
Mapping values into Crogl.....	2
Verifying the setup.....	3

Overview

The Crogl QRadar connector reads and (with explicit approval) writes to IBM QRadar SIEM's REST API (offenses, Ariel/AQL event & flow searches, the asset model, and log sources) using a single **authorized-service token**, sent as the **SEC** header on every call. There is no OAuth flow and no app registration—one token, generated once in the QRadar console, covers every endpoint this connector uses.

Generating that token requires **administrator access to the QRadar console** (a QRadar role that can reach the Admin tab and manage Authorized Services—a QRadar capability, not a Crogl role). If you're an FDE walking a customer through this live, expect their QRadar administrator to drive the console while you confirm the resulting values.

IMPORTANT

The console menu paths and label names below are reconstructed from IBM's published QRadar documentation, not confirmed against a live console while writing this guide. QRadar's Admin UI has also shifted the location of Authorized Services between 7.x releases. If any label or path below doesn't match what the customer sees, trust the live console over this document and update this file afterward. Steps that could not be confirmed are marked **(verify against the customer's console)**.

NOTE

Some on-premises QRadar installs are configured for HTTP Basic authentication instead of a **SEC** token. This connector's default and this guide assume the **SEC** authorized-service token, which is the standard QRadar API-token mechanism. If the customer's environment only supports Basic auth, raise it with the connector maintainer before proceeding—the binding differs.

What kind of account to create

QRadar's model is not an app registration or a dedicated service **user**; it is an **Authorized Service**—a named token that QRadar issues and associates with a **user role** (what it can do) and a **security profile** (what data it can see). Create it from the Admin tab, not from ordinary user management. The token is not tied to a human login session, so it keeps working when the admin who created it is logged out—but it stops working when it reaches its configured expiry (see [Generating the credential](#)).

Permissions to grant

An authorized-service token carries two independent access dimensions—get both right, because a token that authenticates fine can still return no data or a **403** if either is too narrow:

- **User role (capabilities)**. Determines which API operations the token may perform. For this connector the role needs, at minimum: **Offenses** (view/manage—required for the gated offense-note and offense-close write actions as well as reads), **Log Activity**/Ariel search access (for the **aql** event & flow searches), **Assets** (for the asset-model lookups), and a **System Configuration / Log Source management** read capability (for the **list-log-sources** call, which

reads `/api/config/event_sources/...` — a config API distinct from Log Activity, so a token scoped only to Offenses/Ariel/Assets authenticates but returns `403` on log sources). **(Exact capability checkbox names vary by QRadar version — verify against the customer's console.)**

- **Security profile (data visibility).** Determines which domains, log sources, and networks the token can see. In a multi-domain (multi-tenant) QRadar deployment, a security profile scoped to one domain means the token only ever returns that domain's offenses/events — confirm the profile covers the data the customer expects to query. Grant the narrowest profile that still covers the intended scope.

There is no separate admin-consent or approval step as in an OAuth flow; the only gate is that an administrator must have the rights to create the authorized service in the first place.

Generating the credential

(Menu path per IBM documentation — verify against the customer's console.)

1. Sign in to the QRadar console as an administrator.
2. Open the **Admin** tab.
3. Open **Authorized Services** (in some 7.x builds this is under **Admin > User Management > Authorized Services**; in others it is a direct **Authorized Services** item on the Admin tab).
4. Select **Add Authorized Service**.
5. Give it a descriptive name (e.g. `crogl-connector`), then select the **user role** and **security profile** from [Permissions to grant](#).
6. Set an **expiry date**. QRadar offers "No Expiry"; if the customer's policy requires an expiry, note that the token will start returning `401` on that date and must be regenerated and re-entered into Crogl before then.
7. Create the service and **copy the generated token string immediately**. Treat it as shown once — if it's lost, generate a new authorized service rather than trying to recover the value.

You also need the **QRadar console hostname or IP** (no scheme, no path — e.g. `qradar.example.com`). This is the same host an analyst points a browser at for the console UI; the connector calls its REST API at `https://<host>/api`.

Mapping values into Crogl

Two values map to `community/qradar/connector.json`'s `'credential_schema.fields'` when creating the connector instance in Crogl:

Crogl field	Where it came from	Notes
<code>host</code>	The QRadar console hostname or IP	No <code>https://</code> prefix and no path — just the host (e.g. <code>qradar.example.com</code>). Crogl builds the API base URL <code>https://<host></code> for you.

Crogl field	Where it came from	Notes
<code>api_token</code>	Admin > Authorized Services > the generated token string	Treat as a password. Never paste it into chat, a ticket, or this document. Crogl sends it as the SEC header automatically once entered.

If the QRadar console uses a self-signed certificate (common on-premises), enable **skip TLS verification** on the connector when creating it in Crogl; otherwise the connection fails with a TLS handshake error before any request is sent.

Verifying the setup

Right after creating the connector instance in Crogl, confirm the setup:

- **Success:** a simple offense listing (e.g. asking Crogl to "show open offenses") returns **HTTP 200** with offense rows. An empty result on a filtered query is a valid, successful empty response, not an error.
- **AQL query reports "did not complete" on a quiet system:** an event/flow search over a wide window (e.g. **LAST 1 HOURS**) on a low-volume or freshly-stood-up QRadar can exhaust the connector's poll budget before Ariel finishes scanning, and is reported as "did not complete" rather than a result. This is expected on an idle system, not a connector fault — narrow the time range (e.g. **LAST 5 MINUTES**) or point the query at a dataset that has data. A successful AQL run submits (**201**), polls to **COMPLETED**, then fetches results.
- **401:** the token value is wrong, was truncated when copied, or has passed its expiry date — generate a new authorized service in the QRadar console and re-enter the value.
- **403:** the token authenticated, but its **user role or security profile lacks access** to that operation or data — widen the role capability or security-profile scope (see [Permissions to grant](#)) rather than re-checking the token value. This is the most common "right credential, wrong permission" case.
- **404 on /api/siem/offenses:** the **host** value is wrong or includes a scheme/path it shouldn't — it must be a bare hostname/IP.
- **TLS handshake / certificate error before any HTTP status:** the console uses a self-signed certificate and **skip TLS verification** is not enabled on the connector — enable it and retry.